Cuprins

1 Dataset-uri **3**
1.1 Military Aircraft Dataset . . . 3
1.1.1 Structura si organizare . . . 3
1.1.2 Consideratii generale . . . 3
1.2 FGVC Aircraft Dataset . . . 3
1.2.1 Structura si organizare . . . 3
1.2.2 Consideratii generale . . . 4
1.3 MAR20 si Dataset combinat YOLOv8 . . . 4
1.3.1 Colectare si selectie . . . 4
1.3.2 Structura si organizare . . . 5
1.3.3 Consideratii generale . . . 5
1.4 MilitaryAircraftDetectionDataset – EfficientNetB3 . . . 5
1.4.1 Structura si organizare . . . 5
1.4.2 Consideratii generale . . . 5
1.5 Caltech 101 – Vision Transformer . . . 5
1.5.1 Structura si organizare . . . 5
1.5.2 Consideratii generale . . . 6

2 Implementare **7**
2.1 CNN pe Military Aircraft Dataset . . . 7
2.1.1 Procesul prin model . . . 7
2.1.2 Prezentarea dataset-ului . . . 7
2.1.3 Preprocesarea datelor si augmentare . . . 7
2.1.4 Arhitectura modelului . . . 8
2.1.5 Antrenare si validare . . . 8
2.1.6 Evaluarea performantei . . . 8
2.1.7 Vizualizarea predictiilor . . . 9
2.1.8 Concluzii . . . 10
2.1.9 Arhitectura de baza - Desen . . . 11
2.2 DenseNet201 pe FGVC Aircraft . . . 12
2.2.1 Procesul prin model . . . 12
2.2.2 Prezentarea dataset-ului . . . 12
2.2.3 Preprocesarea datelor si augmentare . . . 13
2.2.4 Arhitectura modelului . . . 13
2.2.5 Callbacks pentru antrenare . . . 13
2.2.6 Evaluarea performantei . . . 14
2.2.7 Vizualizare rezultate . . . 14
2.2.8 Concluzii . . . 14

2.2.9 Arhitectura de baza - Desen 14
2.3 EfficientNetB3 pe Military Aircraft Dataset 15
2.3.1 Procesul prin model 15
2.3.2 Prezentarea dataset-ului 15
2.3.3 Preprocesarea datelor si augmentare 16
2.3.4 Arhitectura modelului 16
2.3.5 Callbacks si finetune 16
2.3.6 Antrenare si optimizare 16
2.3.7 Evaluarea performantei 17
2.3.8 Vizualizarea rezultatelor 17
2.3.9 Concluzii 17
2.3.10 Arhitectura de baza - Desen 17
2.4 YOLO11 pe datasetul MAR20 18
2.4.1 Procesul prin model 18
2.4.2 Prezentarea dataset-ului 18
2.4.3 Clasele obiectelor 18
2.4.4 Setup si definirea path-urilor 18
2.4.5 Preprocesarea adnotarilor XML 19
2.4.6 Splitul train/test si conversia la YOLO 19
2.4.7 Fisierul de configurare mar20.yaml 19
2.4.8 Arhitectura YOLO 19
2.4.9 Antrenare YOLO 19
2.4.10 Validare si predictii 19
2.4.11 Concluzii 20
2.4.12 Arhitectura de baza - Desen 20
2.5 YOLOv8 pe dataset combinat 21
2.5.1 Procesul prin model 21
2.5.2 Prezentarea dataset-ului 21
2.5.3 Preprocesarea si combinarea dataset-urilor 22
2.5.4 Antrenare YOLO 22
2.5.5 Predictii si inferenta 22
2.5.6 Reantrenare si finetuning suplimentar 22
2.5.7 Arhitectura de baza - Desen 23
2.6 Vision Transformer pe Aircraft Dataset 24
2.6.1 Procesul prin model 24
2.6.2 Introducere 24
2.6.3 Preprocesarea dataset-ului 24
2.6.4 Patch Creation si Patch Encoding 25
2.6.5 MLP - Multi Layer Perceptron 25
2.6.6 Modelul ViT pentru detectie obiecte 25
2.6.7 Antrenare si evaluare 25
2.6.8 Testare si vizualizare predictii 26
2.6.9 Arhitectura de baza - Desen 26

Chapter 1

Dataset-uri

Toate dataset-urile folosite in acest proiect sunt open source, disponibile public si reutilizabile. Fiecare dataset a fost selectat pentru a oferi date de calitate, structura clara si compatibilitate cu diferite arhitecturi de retele neurale. In continuare, sunt prezentate detaliile fiecarui dataset, modul de organizare si preprocesare, impreuna cu explicatii despre structura fisierelor si folderele aferente.

1.1 Military Aircraft Dataset

1.1.1 Structura si organizare

- Fiecare clasa are propriul folder: B-1, B-2, B-52, C-5, C-17, C-130, C-135, E-3, KC-10 si BareLand.
- Imaginile au dimensiunea aproximativa 600x600 pixeli, cu marimea de 100-200 KB.
- Folderul `sample` contine cate o imagine reprezentativa pentru fiecare clasa.
- Fisierul `TYPE-NAMES.txt` listeaza toate clasele exact in ordinea folosita pentru codificarea etichetelor.

1.1.2 Consideratii generale

- Dataset-ul a fost ales pentru simplitatea structurii si compatibilitatea cu CNN.
- Limitarile includ numarul relativ mic de imagini per clasa si posibile variatii reduse de unghi si iluminare.
- clasa BareLand este minoritara, iar clasele dominante au mai multe exemple; acest dezechilibru poate fi abordat prin augmentare sau re-esantionare.

1.2 FGVC Aircraft Dataset

1.2.1 Structura si organizare

- CSV-uri principale: `train.csv`, `val.csv`, `test.csv` cu coloanele `filename, Classes, Labels`.

- Folderul cu date include:
  - `data/images` – imaginile efective.
  - `families.txt`, `manufacturers.txt`, `variants.txt` – liste cu familii, producatori si variante de aeronave.
  - Fisierele `images_family_train.txt`, `images_family_val.txt`, `images_family_test.txt` definesc split-ul imaginilor pe familii.
  - Fisiere similare pentru `manufacturer` si `variant`.
  - Fisiere de evaluare: `evaluation.m`, `example_evaluation.m` pentru rularea metricilor de performanta.
  - Functii MATLAB suplimentare:
    * `vl_argparse.m` – parsarea argumentelor pentru scripturile de evaluare.
    * `vl_pr.m` – calculul Precision-Recall.
    * `vl_roc.m` – calculul curbei ROC.
    * `vl_tpfp.m` – identificarea True Positives si False Positives.

1.2.2 Consideratii generale

- Dataset-ul este potrivit pentru Fine-Grained Visual Classification datorita clasificarii pe variante, familii si producatori.
- Limitarile includ complexitatea fisierelor si numarul mare de fisiere de index, care poate complica procesul manual de preprocesare.

1.3 MAR20 si Dataset combinat YOLOv8

1.3.1 Colectare si selectie

- Sursele initiale: MAR20, Military Objects, Aircraft Detection (ultimele 2 sunt deja in format yolo, trebuiesc doar agregate).
- Verificarea etichetelor si adnotarilor:
  - Horizontal Bounding Boxes (HBB): dreptunghiuri paralele cu axele imaginii, usor de procesat de YOLO.
  - Oriented Bounding Boxes (OBB): dreptunghiuri rotite pentru a se potrivi cu orientarea reala a avionului, utile pentru obiecte inclinate.
  - OBB ofera mai multa precizie geometrica; HBB este mai simplu si compatibil cu majoritatea framework-urilor.
- Combinarea dataset-urilor presupune realinierea claselor si evitarea duplicatelor.

1.3.2 Structura si organizare

- Folder `Annotations`: fisiere XML cu adnotari pentru fiecare imagine.
- Folder `JpegImages`: imaginile efective.
- Folder `ImageSets`: `train.txt` si `test.txt` cu ID-urile imaginilor pentru split.
- Functii de procesare: `copy_and_rename` si `process_dataset` pentru copiere, redenumire si ajustarea indexului claselor.
- Rezultat final: dataset YOLO unificat cu 115 clase si fisier `data.yaml` continand:
  - caile pentru train/val/test,
  - numarul de clase,
  - lista completa a etichetelor (avioane, vehicule, soldati etc.).

1.3.3 Consideratii generale

- Avantaj: posibilitatea de antrenare unificata YOLOv8 pe mai multe surse.
- Limitare: complexitatea ridicata si necesitatea standardizarii etichetelor.

1.4 MilitaryAircraftDetectionDataset – EfficientNetB3

1.4.1 Structura si organizare

- Folder `annotated_samples`: imagini cu bounding box-uri si clase de avion.
- Folder `crop`: imagini croppate mici pentru fiecare clasa, cu mai multe unghiuri.
- Folder `dataset`: imaginile originale.
- Fisier `labels_with_split.csv` cu coloanele: filename, width, height, class, xmin, ymin, xmax, ymax, split.

1.4.2 Consideratii generale

- Permite antrenarea EfficientNetB3 pe imagini cu bounding box-uri.
- Acces rapid la coordonate si clase.
- Observatii: un echilibru intre clase este important, iar crop-urile multiple ajuta la augmentarea implicita.

1.5 Caltech 101 – Vision Transformer

1.5.1 Structura si organizare

- Toate imaginile relevante sunt intr-un singur folder, airplanes, extras din dataset-ul mare.
-

1.5.2 Consideratii generale

- Limitarile includ variatia redusa a clasei airplanes comparativ cu alte obiecte.
- Structura simpla ajuta la transformarea rapida a imaginilor in patch-uri.

Chapter 2

Implementare

2.1 CNN pe Military Aircraft Dataset

2.1.1 Procesul prin model

Imaginile sunt introduse in CNN, care incepe prin a extrage caracteristicile vizuale folosind straturi convolutionale ce detecteaza margini, texturi si forme. Aceste caracteristici sunt apoi comprimate si agregate prin straturi de pooling, inainte de a fi aplatizate si transmise unui clasificador dens fully connected. Clasificatorul proceseaza aceste reprezentari pentru a determina probabilitatea ca imaginea sa apartina fiecarei clase de avion, rezultand o predictie finala. Intreg procesul invata sa recunoasca tiparele distinctive ale fiecarei clase prin ajustarea ponderilor in timpul antrenarii.

2.1.2 Prezentarea dataset-ului

Dataset-ul utilizat contine imagini cu aeronave militare, elicoptere si drone. Datele au fost organizate astfel:

- **Train** - 70% pentru antrenare;
- **Validation** - 30% pentru validare si ajustarea hiperparametrilor.

B-1	B-2	B-52	C-5	C-17
C-130	C-135	E-3	KC-10	BareLand

Table 2.1: Clasele reprezentative pentru vizualizarea arhitecturii CNN

Fiecare imagine a fost asociata cu clasa corespunzatoare, iar etichetele au fost codificate numeric pentru a fi compatibile cu `CrossEntropyLoss`.

2.1.3 Preprocesarea datelor si augmentare

Transformarile aplicate imaginilor includ:

- Redimensionarea la 130×130 pixeli.
- Flip orizontal aleatoriu pentru antrenare.
- Conversie la *tensor* cu valori in $[0, 1]$.

2.1.4 Arhitectura modelului

Modelul CNN este construit cu doua componente principale:

- **Feature extractor**:
  - Convolutie 3→32 filtre, kernel 3x3, ReLU, MaxPooling 2x2
  - Convolutie 32→64 filtre, kernel 3x3, ReLU, MaxPooling 2x2
  - Convolutie 64→32 filtre, kernel 3x3, ReLU, MaxPooling 2x2
- **Classifier**:
  - Flatten
  - Dense cu 450 neuroni si activare ReLU
  - Dropout 0.65
  - Strat final cu numarul de clase si activare liniara (compatibila cu `CrossEntropyLoss`)

2.1.5 Antrenare si validare

- Optimizer: **Adam** cu learning rate 0.0001
- Functie de pierdere: `CrossEntropyLoss`
- Batch size: 64
- Epoci: 30

La fiecare epoca au fost calculate:

- **Train/Validation Loss** si **Train/Validation Accuracy**

2.1.6 Evaluarea performantei

Modelul a fost evaluat pe setul de validare, iar istoricul metricilor a fost vizualizat prin grafice pentru acuratetea si loss-ul pe train si validation:

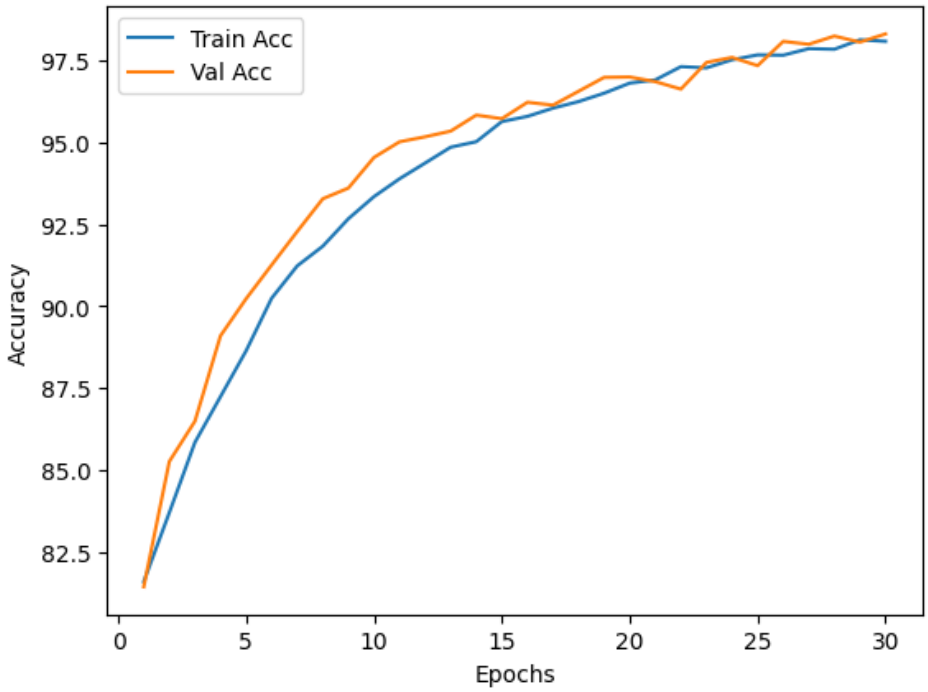

Figure 2.1: Evolutia acuratetii in timpul antrenarii si validarii CNN

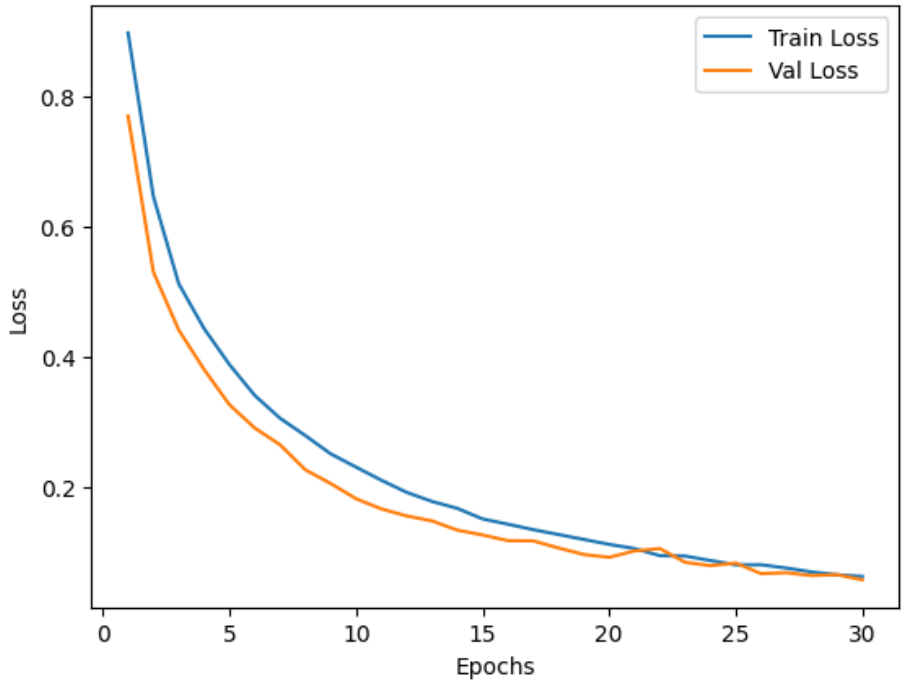

Figure 2.2: Evolutia pierderii (*loss*) in timpul antrenarii si validarii CNN

2.1.7 Vizualizarea predictiilor

Predictiile au fost vizualizate pentru mai multe imagini din fiecare clasa, comparand:

- **T** - clasa reala (*Target*)

- **P** - clasa prezisa (*Predicted*)

Aceasta analiza permite observarea confuziilor frecvente intre clase similare si evaluarea performantelor modelului pe clase rare.

2.1.8 Concluzii

CNN-ul personalizat a oferit o performanta buna pentru recunoasterea multi-clasa a aeronavelor, cu avantajul unei arhitecturi simple si rapide. Limitari si directii de imbunatatire:

- Clasele rare au performanta mai scazuta; se pot testa strategii de *weighted loss*.
- Augmentari suplimentare si ajustarea hiperparametrilor (dropout, learning rate) pot imbunatati generalizarea.
- Modelul CNN poate fi combinat ulterior cu YOLO pentru detectia obiectelor.

2.1.9 Arhitectura de baza - Desen

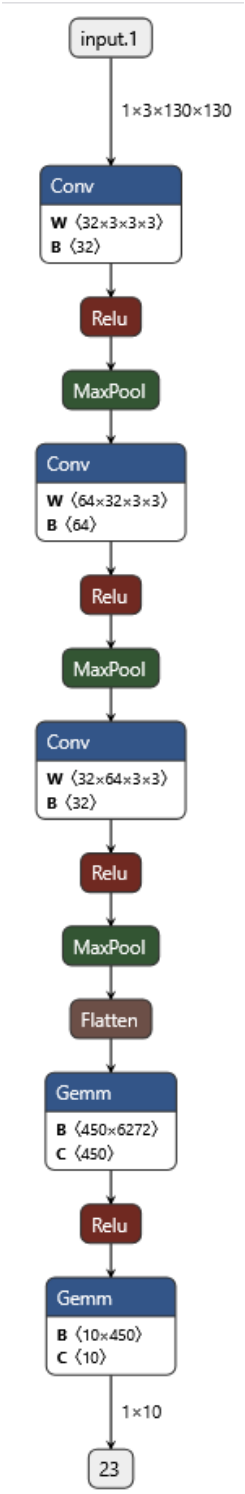

Figure 2.3: Structura retelei CNN: straturi convolutionale, ReLU, MaxPooling, Flatten, Dense si Dropout

2.2 DenseNet201 pe FGVC Aircraft

2.2.1 Procesul prin model

Imaginile sunt mai intai redimensionate si normalizate pentru a se potrivi input-ului din DenseNet201. Fiecare imagine trece printr-un strat initial de convolutie si pooling, care reduce dimensiunile spatiale si extrage caracteristici de baza. Urmeaza mai multe **dense block-uri**, fiecare format din straturi convolutionale interconectate: output-ul fiecarui strat este concatenat cu output-urile tuturor straturilor anterioare din bloc. Astfel se face propagarea eficienta a caracteristicilor si gradientului, pastrand detalii esentiale pentru recunoasterea obiectelor similare. Dupa fiecare dense block urmeaza un **transition layer** cu convolutie si pooling pentru a reduce dimensiunea si a evita supraincarcarea. Output-ul final este agregat prin **Global Average Pooling**, normalizat si trecut prin straturi dense cu activare Mish si dropout, care invata sa distinga clasele.

2.2.2 Prezentarea dataset-ului

Dataset-ul utilizat este **FGVC Aircraft**, un set de date destinat recunoasterii fine-grained a aeronavelor. Acesta contine aproximativ 10.000 de imagini impartite in:

- **Train** - pentru antrenarea modelului si contine 100 de clase;
- **Validation** - pentru selectarea hiperparametrilor si prevenirea overfitting-ului;
- **Test** - pentru evaluarea finala.

100	135	145	146-200	146-300
172	1900	2000	208	328
340	50	525	560	600
70	707-320	717	727-200	737-200
737-300	737-400	737-500	737-600	737-700
737-800	737-900	747-100	747-200	747-300
747-400	757-200	757-300	767-200	767-300
767-400	777-200	777-300	900	A300B4
A310	A318	A319	A320	A321
A330-200	A330-300	A340-200	A340-300	A340-500
A340-600	A380	ATR-42	ATR-72	An-12
B200	BAE	BAE-125	Beechcraft	Boeing
C-130	C-47	CRJ-200	CRJ-700	CRJ-900
Cessna	Challenger	DC-10	DC-3	DC-6
DC-8	DC-9-30	DH-82	DHC-1	DHC-6
DHC-8-100	DHC-8-300	DR-400	Dornier	E-170
E-190	E-195	EMB-120	ERJ	Embraer
Eurofighter	Express	F-16A/B	F/A-18	Falcon
Fokker	Global	Gulfstream	Hawk	IV
Il-76	L-1011	Legacy	MD-11	MD-80
MD-87	MD-90	Metroliner	Model	PA-28
SR-20	Saab	Spitfire	T1	Tornado
Tu-134	Tu-154	Typhoon	V	Yak-42

Table 2.2: Lista claselor din dataset-ul FGVC Aircraft

Fiecare imagine are asociata o clasa si un index numeric. Dataset-ul contine peste 100 de tipuri distincte de aeronave comerciale si militare, incluzand serii Boeing, Airbus, Embraer, Bombardier, avioane clasice si aeronave de mici dimensiuni. (Tabel 1.1)

Pentru clasificare multi-clasa, etichetele au fost convertite in reprezentare *one-hot encoding*, astfel incat functia `categorical_crossentropy` sa poata fi utilizata.

2.2.3 Preprocesarea datelor si augmentare

Fiecare imagine este:

- Citita si normalizata in intervalul $[0, 1]$.
- Redimensionata la $299 \times 299 \times 3$, conform input-ului DenseNet201.

Pentru imbunatatirea generalizarii, a fost aplicata augmentare de date:

- Flip orizontal si vertical aleatoriu.
- Variatii de luminozitate, contrast, saturatie si nuanta.
- Rotatii la $90°$ aleatorii.

Pipeline-ul a fost implementat folosind `tf.data.Dataset` cu `AUTOTUNE`, ceea ce permite rularea paralela si prefetching pentru eficienta maxima pe GPU.

2.2.4 Arhitectura modelului

DenseNet201, preantrenat pe ImageNet, a fost folosit ca baza (*feature extractor*) fara stratul top. Peste acesta au fost adaugate:

- `GlobalAveragePooling2D` pentru reducerea dimensiunii spatiale.
- Straturi `Dense` cu activare *mish*, regularizate cu `l2`.
- `BatchNormalization` pentru stabilizarea antrenarii.
- `Dropout` (0.7 si 0.6) pentru reducerea overfitting-ului.
- Strat final `Dense` cu 100 neuroni si activare `softmax`.

Optimizerul folosit a fost **SGD** cu scheduler pentru rata de invatare.

2.2.5 Callbacks pentru antrenare

Au fost implementate:

- `CyclicLR` - scheduler ciclic la learning rate pentru a iesi din minime locale.
- `ReduceLROnPlateau` - reducerea automata a learning rate cand validarea nu mai scade.
- `EarlyStopping` - oprirea antrenarii cand nu se mai imbunatatesc performantele.
- `ModelCheckpoint` - salvarea celui mai bun model obtinut.

Antrenarea s-a facut pe Batch-uri de **16 imagini**, timp de 40 de epoci initial, cu posibilitatea continuarii pe epoci suplimentare.

2.2.6 Evaluarea performantei

La final, modelul a fost evaluat pe setul de test:

- **Acuratete pe test**: aproximativ 80%
- A fost calculat **classification report** ce are: *precision*, *recall*, *f1-score*.
- A fost generat **confusion matrix** pentru a observa erorile intre clase similare.

2.2.7 Vizualizare rezultate

Au fost selectate imagini din setul de test cu:

- **Predictii corecte** - confirmand ca modelul poate recunoaste clasele distincte.
- **Predictii gresite** - utile pentru analiza erorilor si intelegerea dificultatilor datasetului (ex. confuzii intre aeronave similare).

2.2.8 Concluzii

DenseNet201 s-a dovedit un model performant pentru clasificarea aeronavelor, cu o buna capacitate de generalizare. In schimb, urmatoarele lucruri au ramas de imbunatatit:

- Clasele similare sunt inca dificil de separat.
- Performanta finala depinde de optimizarea hiperparametrilor (dropout, lr, batch size, etc.). Trebuie facute mai multe teste si/sau scenarii cu mai multe valori

2.2.9 Arhitectura de baza - Desen

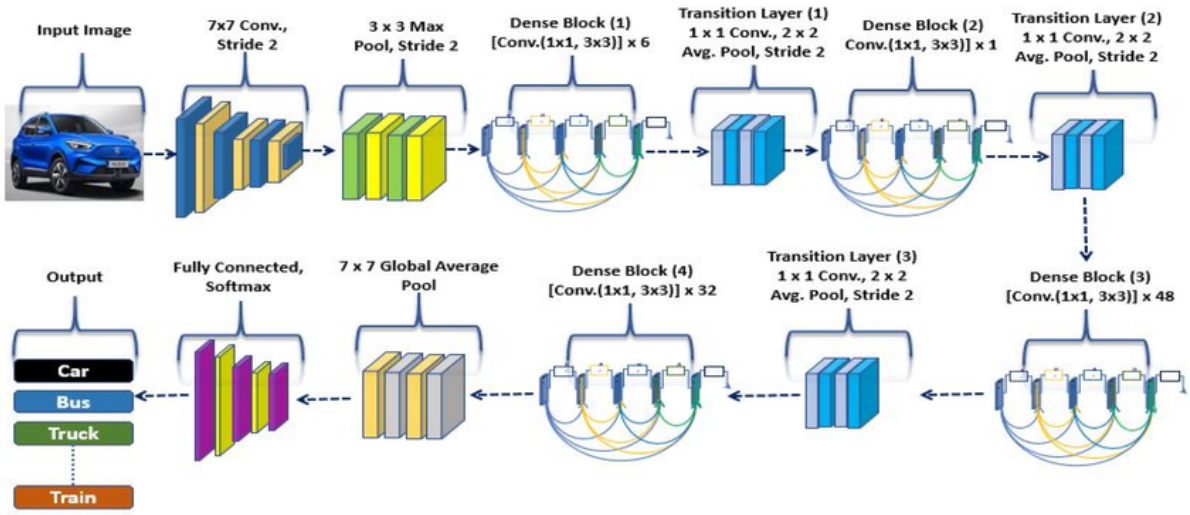

Figure 2.4: Arhitectura DenseNet201: input image → initial convolution + pooling → dense block-uri intercalate cu transition layers (conv + pooling) → global average pooling → fully connected layer pentru clasificare. In fiecare dense block, fiecare layer primeste ca input toate output-urile layere-lor anterioare din acel block (conexiuni dense).

2.3 EfficientNetB3 pe Military Aircraft Dataset

2.3.1 Procesul prin model

Imaginile redimensionate la $224 \times 224 \times 3$ intra in blocurile MBConv ale EfficientNetB3, care aplica convolutii depthwise si pointwise combinate cu **Squeeze-and-Excitation** pentru a ajusta importanta canalelor. Fiecare bloc include normalizare batch si activare Swish, urmand o schema de downsampling pentru reducerea dimensiunii spatiale gradual. Caracteristicile extrase trec prin **Global Average Pooling** pentru a obtine un vector de caracteristici compact. Acesta este trecut prin straturi dense cu regularizare L2 si dropout, care combina si filtreaza informatiile relevante pentru clasificarea pe clase. Modelul invata relatiile complexe intre caracteristici locale si globale, optimizand ponderile convolutiilor si dense-urilor prin backpropagation cu Adam.

2.3.2 Prezentarea dataset-ului

Dataset-ul utilizat contine **95 de clase** de aeronave militare, elicoptere si drone. Acesta a fost filtrat pentru a elimina clasele irelevante si pentru a reduce dezechilibrul intre clase. Datele au fost impartite astfel:

- **Train** - 80% din date pentru antrenarea modelului;
- **Validation** - 10% pentru ajustarea hiperparametrilor;
- **Test** - 10% pentru evaluarea finala.

A10	A400M	AG600	AH64	AKINCI
AV8B	An124	An22	An225	An72
B1	B2	B21	B52	Be200
C1	C130	C17	C2	C390
C5	CH47	CH53	CL415	E2
E7	EF2000	EMB314	F117	F14
F15	F16	F18	F2	F22
F35	F4	FCK1	H6	Il76
J10	J20	J35	J36	J50
JAS39	JF17	JH7	KAAN	KC135
KF21	KJ600	Ka27	Ka52	MQ25
MQ9	Mi24	Mi26	Mi28	Mi8
Mig29	Mig31	Mirage2000	P3	RQ4
Rafale	SR71	Su24	Su25	Su34
Su47	Su57	TB001	TB2	Tejas
Tornado	Tu160	Tu22M	Tu95	U2
UH60	US2	V22	V280	Vulcan
WZ10	WZ7	WZ9	X29	X32
XB70	Y20	YF23	Z10	Z19

Table 2.3: Lista claselor filtrate pentru detectia aeronavelor militare, elicopterelor si dronelor

2.3.3 Preprocesarea datelor si augmentare

Fiecare imagine a fost:

- Citita si redimensionata la $224 \times 224 \times 3$, conform input-ului EfficientNetB3.
- Normalizata in intervalul $[0, 1]$.

Pentru a reduce overfitting-ul si a creste capacitatea de generalizare, s-au aplicat urmatoarele augmentari:

- Flip aleator orizontal.
- Rotatie aleatorie de maxim 10%.
- Zoom aleatoriu de maxim 10%.

Datele au fost incarcate folosind `tf.data.Dataset` cu `cache`, `shuffle`, `batch` si `prefetch` pentru a maximiza utilizarea GPU-ului.

2.3.4 Arhitectura modelului

EfficientNetB3, preantrenat pe ImageNet, a fost folosit ca extractor de caracteristici (*feature extractor*) cu stratul top eliminat. Modelul final a fost construit astfel:

- Straturi de augmentare a datelor (`RandomFlip`, `RandomRotation`, `RandomZoom`).
- `BatchNormalization` pe canalele finale pentru stabilitate.
- `GlobalAveragePooling2D` pentru reducerea dimensiunii caracteristicilor.
- Strat complet conectat (`Dense`) cu 512 neuroni si regularizare L2.
- `Dropout` 0.4 pentru prevenirea overfitting-ului.
- Strat final `Dense` cu 95 neuroni si activare `softmax`.

2.3.5 Callbacks si finetune

Pentru antrenare, au fost utilizate urmatoarele mecanisme:

- `ModelCheckpoint` pentru salvarea celui mai bun model.
- `ReduceLROnPlateau` pentru ajustarea automata a learning rate-ului.
- `EarlyStopping` pentru oprirea antrenarii cand validarea nu mai imbunatateste performanta.

2.3.6 Antrenare si optimizare

Modelul a fost antrenat cu:

- Optimizer **Adam** cu learning rate 0.001.
- Functie de pierdere `categorical_crossentropy`.
- Batch-uri de 32 imagini.
- 50 epoci initial, cu posibilitatea de a continua inca 20 epoci.

2.3.7 Evaluarea performantei

Performanta modelului a fost evaluata pe setul de test:

- **Acuratete finala**: 91%.
- Calculul **precision**, **recall** si **f1-score** pentru fiecare clasa.
- Generarea **confusion matrix** pentru vizualizarea erorilor intre clase similare.

2.3.8 Vizualizarea rezultatelor

Au fost afisate imagini cu predictii corecte si gresite, pentru analiza tipurilor de confuzie. De ex:

- Confuzii frecvente intre avioane de acelasi tip (ex. F16 vs Rafale).
- Recunoasterea corecta a elicopterelor si dronelor.

2.3.9 Concluzii

EfficientNetB3 a oferit o performanta decenta avand in vedere numarul de clase si distrubutia dataset-ului. Totusi, mai raman de imbunatatit sau aplicat urmatoarele:

- Distributia neuniforma a claselor ramane o provocare.
- Testarea unor strategiile de weighted loss ar putea imbunatati performanta pe clasele rare.

2.3.10 Arhitectura de baza - Desen

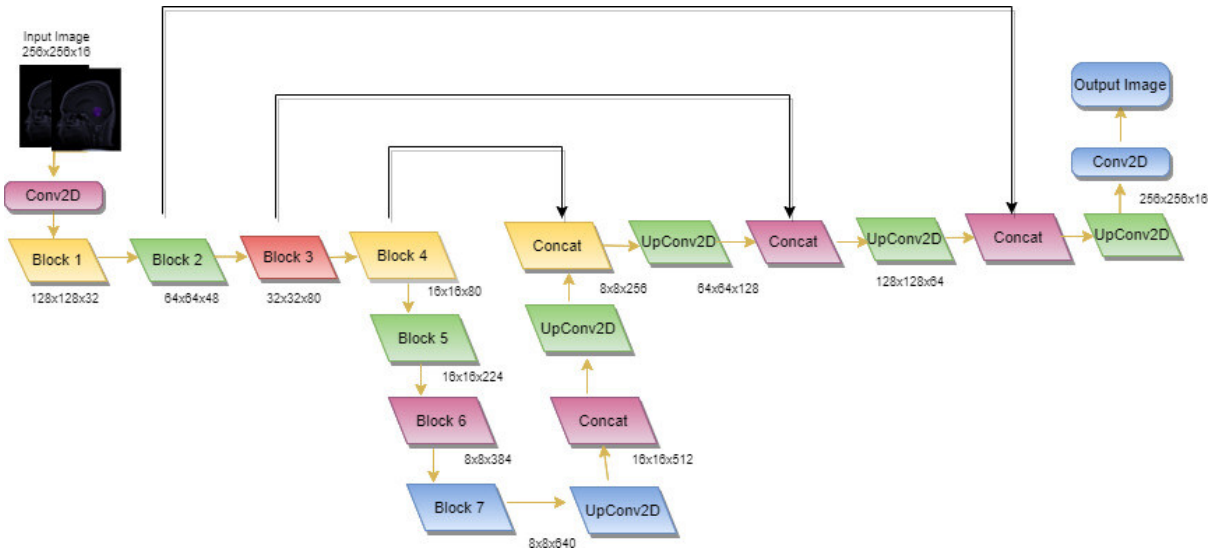

Figure 2.5: Structura modelului EfficientNetB3 cu straturi de augmentare, pooling, dense si dropout

2.4 YOLO11 pe datasetul MAR20

2.4.1 Procesul prin model

Imaginile MAR20 intra in YOLO11, unde **backbone-ul** (ex. Darknet sau CSP) extrage caracteristici vizuale folosind straturi convolutionale cu downsampling progresiv, captand forme, margini si texturi. **Neck-ul** (Feature Pyramid Network) combina caracteristicile la multiple scale-uri pentru detectarea obiectelor mici si mari simultan. **Head-ul** genereaza pentru fiecare grila probabilitatile claselor si coordonatele bounding box-urilor folosind ancore predefinite. Modelul calculeaza loss-uri combinate: clasa, pozitie si scor de incredere si ajusteaza ponderile prin backpropagation pentru a minimiza diferenta fata de ground-truth. In timpul inferentei, imaginile trec rapid prin pipeline-ul backbone-neck-head, rezultand predictii bounding box si confidence score pentru fiecare obiect detectat.

2.4.2 Prezentarea dataset-ului

Dataset-ul MAR20 contine imagini cu avioane, fiecare avand adnotari in format Pascal VOC (XML). Structura dataset-ului include:

- **JPEGImages** - imagini;
- **Annotations** - fisiere XML cu bounding box-uri si clase;
- **ImageSets/Main** - spliturile *train* si *test*.

2.4.3 Clasele obiectelor

Dataset-ul contine 20 de clase, indexate numeric, utilizate in YOLO:

A1	A10	A11	A12	A13
A14	A15	A16	A17	A18
A19	A2	A20	A3	A4
A5	A6	A7	A8	A9

Table 2.4: Clasele obiectelor din MAR20 extrase din fisierul mar20.yaml

2.4.4 Setup si definirea path-urilor

S-au definit directoarele pentru:

- Imagini: `IMAGES_DIR`
- Adnotari: `ANNOT_ROOT`
- Splituri train/test: `SPLIT_DIR`
- Workspace YOLO: `WORK`
- Fisier YAML de configurare: `DATA_YAML`
- Predictii YOLO: `PRED_DIR`

2.4.5 Preprocesarea adnotarilor XML

Fisierele XML au fost mapate la imaginile corespunzatoare, selectand pentru fiecare imagine fisierul cel mai complet. Bounding box-urile au fost verificate si convertite in format YOLO, cu valori normalizate $[0, 1]$.

2.4.6 Splitul train/test si conversia la YOLO

- Id-urile imaginilor pentru train si test au fost citite din `train.txt` si `test.txt`.
- Folderele `images/train`, `images/test`, `labels/train`, `labels/test` au fost create.
- Adnotarile XML au fost convertite in fisiere `.txt` compatibile YOLO.

2.4.7 Fisierul de configurare mar20.yaml

Fisierul YAML contine:

- Path-ul principal al dataset-ului YOLO
- Locatiile imaginilor si etichetelor pentru train, test si val
- Lista claselor cu indecsii lor (0-19)

2.4.8 Arhitectura YOLO

YOLO utilizeaza o retea de tip *backbone + neck + head*:

- **Backbone** - extrage caracteristici de la imaginile de intrare (Darknet, CSP, sau EfficientNet in YOLOv8)
- **Neck** - feature pyramid pentru detectia obiectelor la multiple scale-uri
- **Head** - genereaza bounding box-uri, scoruri si clase pentru fiecare obiect detectat

2.4.9 Antrenare YOLO

- Model preantrenat: `yolo11x.pt` sau fallback `yolov8n.pt`
- Parametrii antrenarii: `epochs=50`, `imgsz=640`, `device=0`, batch automat
- Early stopping activat prin `patience=15`
- Modelul este antrenat cu imaginile si adnotarile YOLO convertite

2.4.10 Validare si predictii

- Validarea modelului a fost realizata pe split-ul *val* folosind `model.val`
- Predictiile au fost generate pe setul de test cu salvarea imaginilor si a fisierelor `.txt` in `PRED_DIR/yolo_preds`
- Rezultatele permit vizualizarea bounding box-urilor prezise si compararea cu ground-truth

2.4.11 Concluzii

Implementarea YOLO pe dataset-ul MAR20 permite:

- Detectia multi-clasa a aeronavelor si elicopterelor
- Vizualizarea bounding box-urilor pentru analiza performantelor
- Posibila integrare ulterioara cu modelul CNN pentru clasificare complementara

2.4.12 Arhitectura de baza - Desen

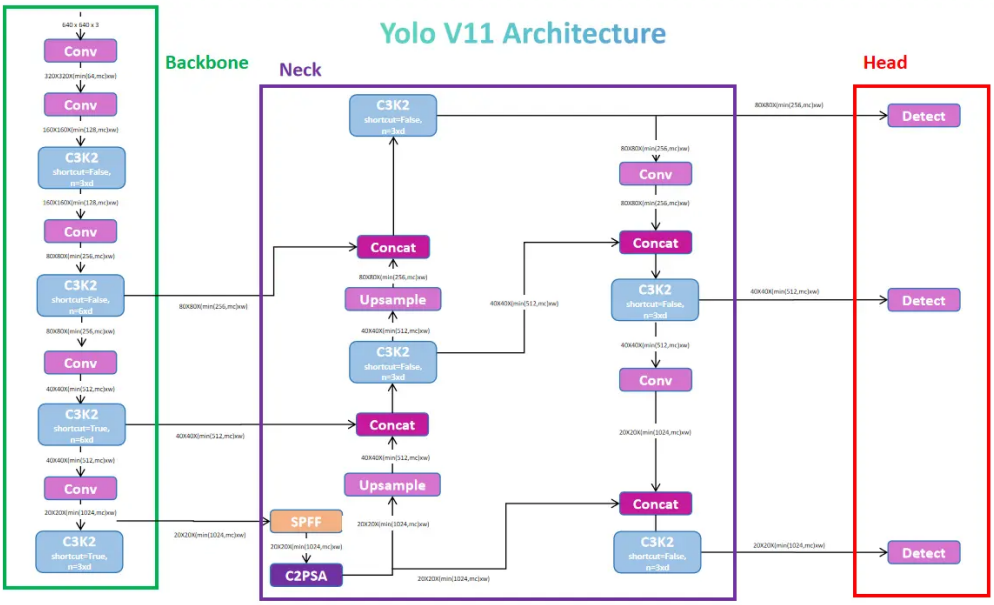

Figure 2.6: Structura detaliata a pipeline-ului YOLO pentru dataset-ul MAR20: backbone, neck si head pentru detectia obiectelor

2.5 YOLOv8 pe dataset combinat

2.5.1 Procesul prin model

Imaginile combinate sunt procesate astfel: **backbone-ul** (CSP/Darknet modificat sau EfficientNet) extrage caracteristici la niveluri diferite de detaliu. **Neck-ul** construieste o piramida de caracteristici pentru a integra informatia de la toate scale-urile, folosind concateneri si upsampling pentru a pastra detalii atat locale cat si globale. **Head-ul** aplica convolutii si ancore pentru fiecare grila, generand predictii de bounding box, scor de incredere si probabilitate pentru fiecare dintre cele 115 clase. Ponderile sunt optimizate cu AdamW si augmentarile din pipeline permit generalizarea pe obiecte variate. La inferenta, imaginile trec prin backbone-necks-head pentru a produce bounding box-uri si probabilitati precise pentru toate obiectele din view.

2.5.2 Prezentarea dataset-ului

Din prelucrarea dataset-urilor, au fost extrase urmatoarele clase:

Mi28	UH60	F16	H6	F18
F22	F35	JAS39	B52	JH7
KF21	US2	Be200	AG600	EF2000
C2	P3	CL415	E2	Rafale
Mig31	Su25	XB70	YF23	Su57
F15	KC135	An72	An22	B2
An124	Tu160	C5	Il76	J20
B1	C130	A10	V22	C390
F117	WZ7	E7	CH47	AH64
AV8B	Mi24	U2	KJ600	RQ4
Tu22M	SR71	J10	Mirage2000	Tu95
A400M	MQ9	F14	Su34	F4
Mi8	Mig29	Z10	JF17	TB001
KAAN	Ka52	C17	EMB314	An225
Y20	Su24	TB2	Ka27	Tornado
Vulcan	Z19	V280	Mi26	J36
J35	B21	WZ9	camouflage_soldier	weapon
military_tank	military_truck	military_vehicle	civilian	soldier
civilian_vehicle	military_artillery	trench	military_aircraft	military_warship
A1	A10	A11	A12	A13
A14	A15	A16	A17	A18
A19	A2	A20	A3	A4
A5	A6	A7	A8	A9

Table 2.5: Clasele incluse in dataset-ul combinat YOLO (115 clase)

Pentru antrenarea YOLO a fost creat un dataset unificat pornind de la trei surse:

- **Aircraft Detection** - 83 clase
- **Military Object Dataset** - 12 clase

- **MAR20** - 20 clase

Dataset-ul final contine imagini pentru train, val si test, cu **115 clase** distincte, incluzand avioane, elicoptere, vehicule si soldati. Structura dataset-ului este compatibila YOLO, cu foldere pentru `images` si `labels`.

2.5.3 Preprocesarea si combinarea dataset-urilor

- Imaginile si etichetele din cele trei surse au fost copiate intr-un director unificat.
- Fisierele au fost redenumite pentru a evita coliziuni de nume.
- Clasele au fost realiniate si indexate pentru consistenta, astfel incat YOLO sa le poata interpreta corect.
- S-a generat fisierul `data.yaml` cu calea catre folderele train/val/test si lista completa a claselor.

2.5.4 Antrenare YOLO

Modelul preantrenat `yolov8m.pt` a fost folosit pentru finetuning pe dataset-ul combinat:

- **Epochs**: 50
- **Batch size**: 16
- **Img size**: 256
- **Optimizer**: AdamW
- **Early stopping**: 10 epoci fara imbunatatire
- **Half precision**: True (accelerare GPU)

Validarea s-a efectuat pe split-ul `val`, monitorizand metrice precum **mAP**, **precizie** si **recall**.

2.5.5 Predictii si inferenta

- Dupa antrenare, modelul optim (`best.pt`) a fost incarcat pentru inferenta.
- Pentru test, s-au selectat imagini random din folderul `test/images`.
- S-au rulat predictii YOLO, afisand bounding box-urile pentru obiectele detectate.
- Predictiile au fost salvate in `predictions/yolo_preds`, cu fisiere .jpg si .txt.

2.5.6 Reantrenare si finetuning suplimentar

- Modelul poate fi reluat de la un checkpoint anterior (`best.pt`) pentru continuarea antrenarii.
- S-au folosit callback-uri pentru reducerea learning rate-ului pe baza performantei (`ReduceLROnPlateau`).
- Parametrii suplimentari: epochs 100, batch 32, imgsz 320, optimizer SGD, half precision True.

2.5.7 Arhitectura de baza - Desen

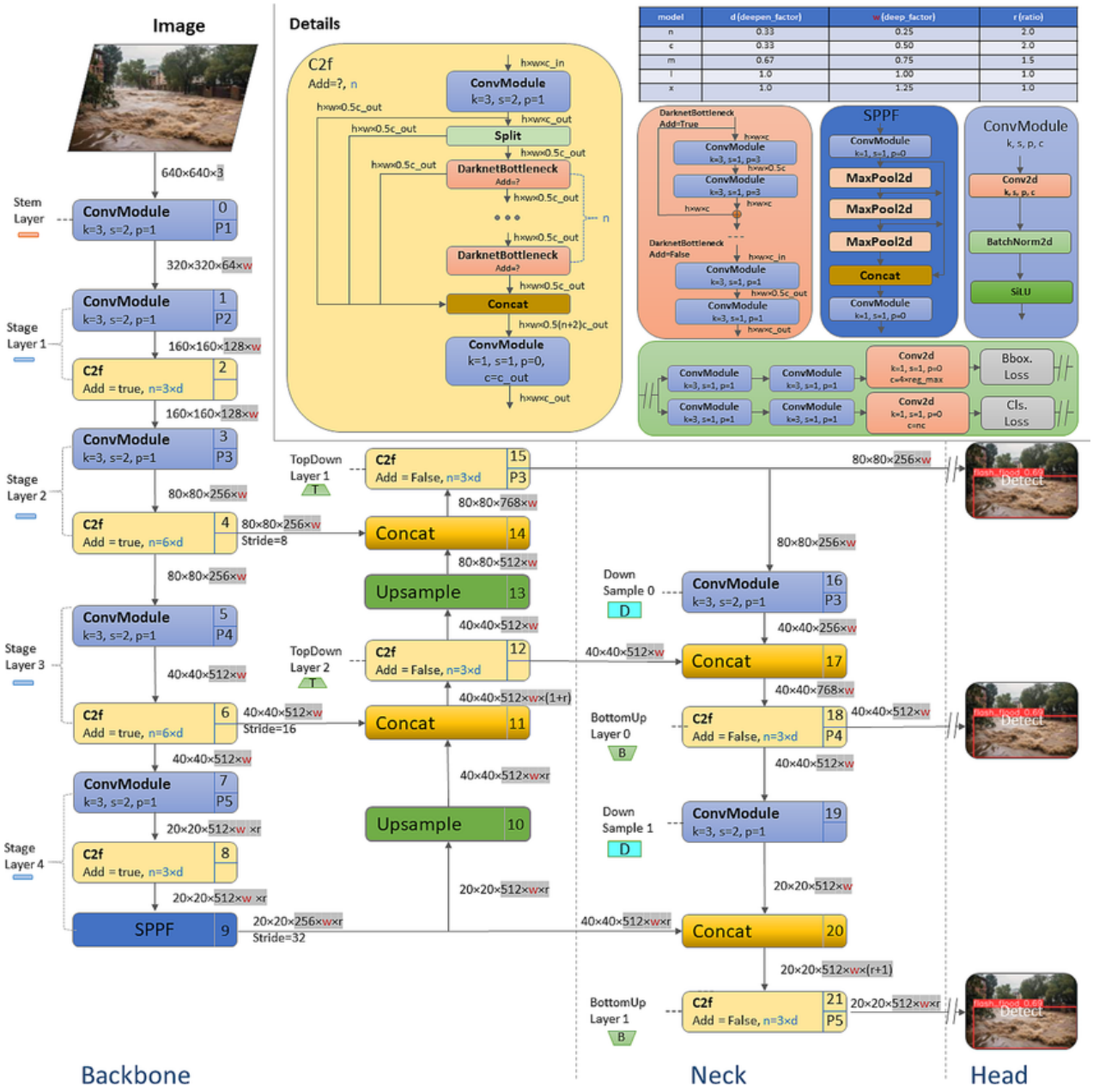

Figure 2.7: Structura pipeline-ului YOLO pe dataset-ul combinat: backbone, neck si head

2.6 Vision Transformer pe Aircraft Dataset

2.6.1 Procesul prin model

Imaginile 224×224 sunt impartite in patch-uri 32×32, fiecare transformat intr-un embedding liniar si combinat cu **positional embeddings** pentru a pastra informatia de pozitie. Patch-urile sunt trecute prin mai multe **straturi Transformer**, fiecare continand Multi-Head Attention si Layer Normalization cu skip connections. In Multi-Head Attention, embedding-urile patch-urilor sunt proiectate in query, key si value, calculand scorurile de atentie pentru fiecare head si combinandu-le pentru a captura relatiile locale si globale intre patch-uri. Output-ul este flatten-uit si transmis printr-un MLP cu mai multe straturi dense si activare GELU, care prezice coordonatele bounding box $(x_{min}, y_{min}, x_{max}, y_{max})$. Modelul optimizeaza ponderile prin AdamW pentru a minimiza MSE intre coordonatele prezise si cele reale, combinand informatia locala din patch-uri cu contextul global al imaginii pentru o detectare precisa a obiectului.

2.6.2 Introducere

In acest experiment am utilizat un **Vision Transformer (ViT)** pentru detectia avioanelor intr-o imagine, folosind dataset-ul **Caltech 101**. Scopul modelului este de a prezice coordonatele unui **bounding box** care incadreaza avionul. Vision Transformer combina mecanismul de *self-attention* cu procesarea secventiala a patch-urilor extrase din imagine, permitand modelului sa invete atat caracteristici locale, cat si dependente globale intre regiuni.

2.6.3 Preprocesarea dataset-ului

Dataset-ul **Caltech 101** contine imagini pentru 101 clase diferite, dintre care am selectat doar imaginile cu obiecte de tip **airplanes**. Pentru fiecare imagine, bounding box-ul avionului a fost preluat din fisierele de adnotare corespunzatoare.

- Imaginile au fost citite din folderul `101_ObjectCategories/airplanes/`.
- Bounding box-urile au fost extrase din fisierele `Annotations/Airplanes_Side_2/`.
- Toate imaginile au fost redimensionate la 224×224 pixeli pentru a se potrivi dimensiunilor de input ale modelului ViT.
- Coordonatele bounding box-urilor au fost normalizate in intervalul $[0, 1]$, proportional cu dimensiunea imaginii.
- Datele au fost convertite in array-uri Numpy si impartite in seturi de antrenament si test, cu un split de 80%-20%.

Aceasta etapa de preprocesare este cruciala, deoarece ViT proceseaza secvente de patch-uri extrase din imagini, iar normalizarea si redimensionarea asigura consistenta datelor de intrare.

2.6.4 Patch Creation si Patch Encoding

- **Layer-ul Patches**: imparte imaginea in patch-uri de dimensiune 32×32 pixeli. Fiecare patch contine informatii locale despre obiecte si textura.
- **Layer-ul PatchEncoder**: transforma fiecare patch intr-un vector de dimensiune `projection_dim` si adauga informatii despre pozitia patch-ului (*positional embeddings*) pentru ca modelul sa invete structura spatiala.

Patch-urile sunt apoi trimise printr-o serie de layere Transformer pentru a invata atat relatii locale intre patch-uri cat si dependente globale intre regiuni diferite ale imaginii.

2.6.5 MLP - Multi Layer Perceptron

- Fiecare patch procesat prin Transformer este trecut printr-un MLP fully connected.
- MLP-ul contine mai multe hidden layers cu activare GELU si dropout pentru regularizare.
- Output-ul MLP-ului reprezinta coordonatele bounding box-ului pentru avion: $(x_{min}, y_{min}, x_{max}, y_m$
- Dropout-ul ajuta la prevenirea supra-antrenarii si imbunatateste generalizarea.

2.6.6 Modelul ViT pentru detectie obiecte

- **Input:** imagine $224 \times 224 \times 3$.
- **Patch-uri:** 32×32, generand un total de $(224/32)^2 = 49$ patch-uri per imagine.
- **Encoder:** 4 layere Transformer, fiecare cu Multi-Head Attention, Layer Normalization si skip connections.
- **Output:** patch-urile encodate sunt flatten-uite si trecute printr-un MLP pentru a prezice coordonatele bounding box-ului.

Aceasta arhitectura permite modelului sa combine informatia locala (patch-uri) cu informatia globala (relatii intre patch-uri) pentru detectia precisa a obiectului.

2.6.7 Antrenare si evaluare

- **Optimizer:** AdamW cu *weight decay* pentru regularizare suplimentara.
- **Loss:** Mean Squared Error (MSE) intre coordonatele prezise si cele reale.
- **Early stopping:** monitorizare a pierderii pe setul de validare si salvarea celui mai bun model.
- **Checkpointing:** salvarea modelului cu cea mai mica pierdere de validare.
- Evaluare: **Intersection over Union (IoU)** pentru bounding box-urile prezise.

2.6.8 Testare si vizualizare predictii

- S-au selectat 15 imagini random din setul de test.
- Pentru fiecare imagine s-au afisat side-by-side: bounding box-ul prezis (rosu) si bounding box-ul real (verde).
- Media IoU a fost calculata pentru a cuantifica performanta modelului.

2.6.9 Arhitectura de baza - Desen

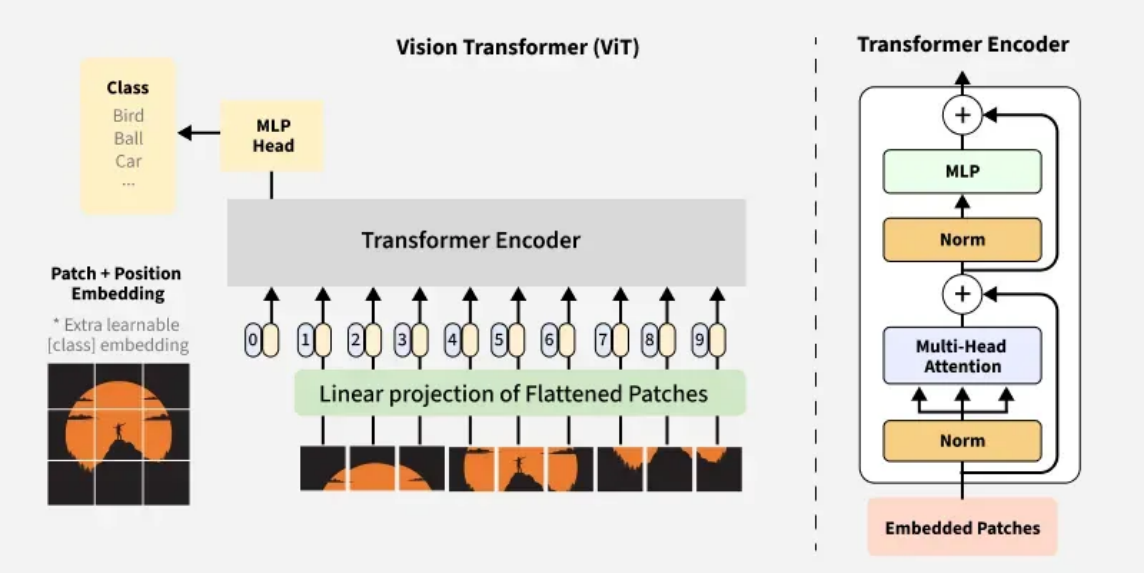

Figure 2.8: Arhitectura Vision Transformer pentru detectia avioanelor: Input image → Patch Creation → Patch Encoder → Transformer Layers (MultiHeadAttention + Skip Connections) → Flatten → MLP → Bounding Box output.

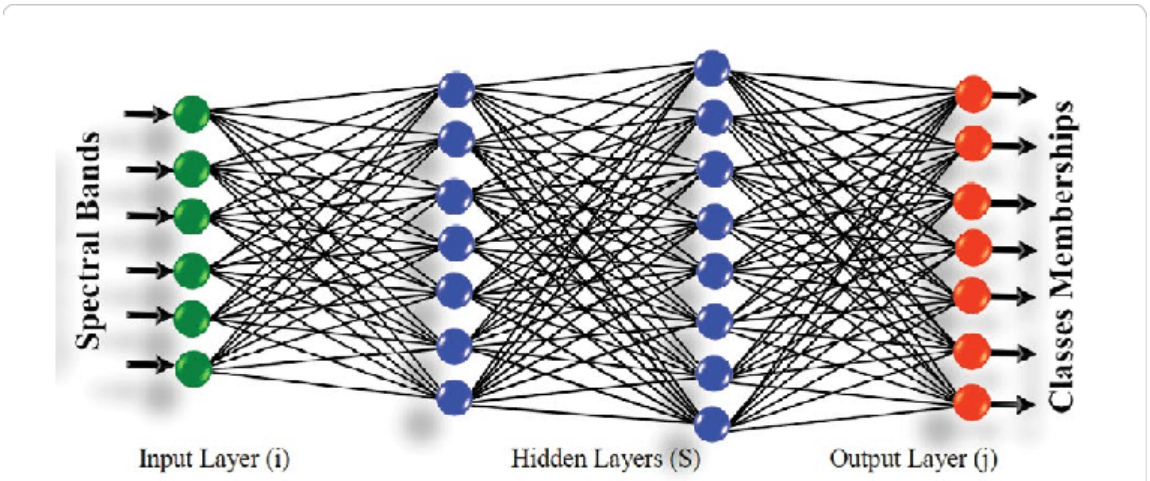

Figure 2.9: MLP Head folosit pentru predictia bounding box-ului: multiple fully connected layers cu activare GELU si dropout.

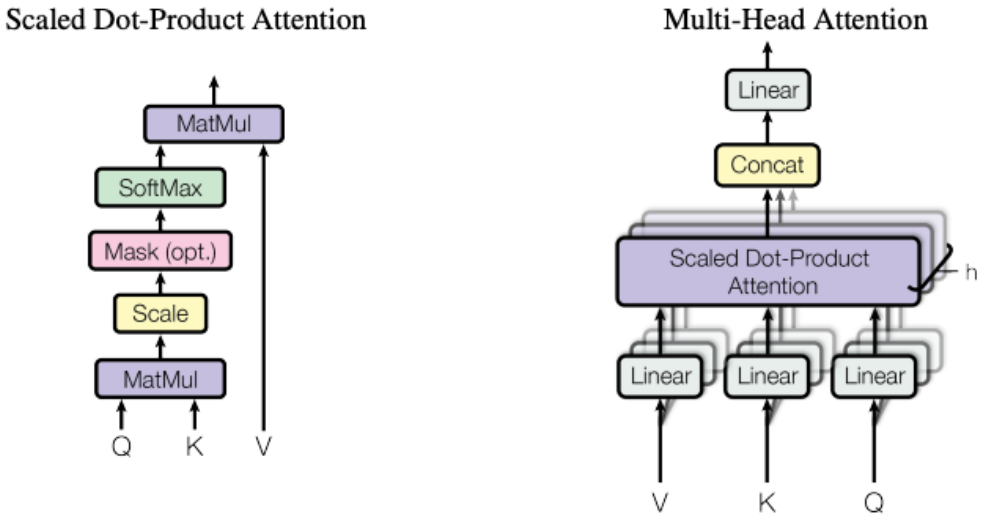

Figure 2.10: Structura mecanismului Multi-Head Attention: input embeddings sunt proiectate in query (Q), key (K) si value (V). Pentru fiecare head, se calculeaza produsul QK^T, acesta este scalat prin $\frac{1}{\sqrt{d_k}}$ pentru stabilitate numerica si apoi se aplica functia softmax pentru a obtine ponderile de atentie. Rezultatul ponderat se inmulteste cu V, iar toate head-urile sunt concatenate si trecute printr-un layer de proiectie liniara final.